

아래 문서는 새 채팅창에서 그대로 붙여넣어 “컨텍스트 시드”로 쓰기 좋은 _업무 인수인계 보고서_입니다.

(새 채팅은 기존 기록을 자동으로 기억하지 않으므로, 이 문서를 첫 메시지로 붙여 시작해 주세요.)

컬리저널 Admin — 업무 인수인계 보고서 (핵심 정리)

0) 현재 상태 요약 (Done / To-do)

- ✓ Cloud Run 배포 정상: /health 200, /api/status 200(헤더 X-Admin-Token 필요).
- ✓ 오케스트레이터 포함: Dockerfile에서 orchestrator.py 및 admin/·tools/ 명시적 COPY.
- ✓ 트래픽 100% 최신 리버전: --traffic latest=100 및 update-traffic --to-latest 적용.
- ✓ 버튼 무반응 이슈 해결: btnRefresh → loadReady(), btnKpiRefresh → refresh() 맥등 바인딩.
- ✓ syncFallback if-else 사다리 깨짐 수정: 하나의 연속 사다리로 정리.
- ✓ 게이트 슬라이더 UX 개선: oninput에서 gatePass 즉시 반영.
- ✓ 게이트 저장 API 확인: PATCH /api/config/gate_required 바디 {"gate_required": n} 로 200 확인.
- ✓ “수집만” 서버 동작 확인: POST /api/flow/keyword + {"collect_only": true} 로 200 확인.
- ▶ To-do #1 (프론트): “수집만” 버튼이 아직 /api/flow/keyword/collect 를 부를 수 있어, 경로를 /api/flow/keyword 로 바꾸고 바디에 {collect_only:true} 추가 필요.
- ▶ To-do #2 (프론트): updateGate()가 {"value":n}을 보낸다면 **{"gate_required":n}**로 변경 필요.

1) 운영 환경·경로

1.1 폴더 구조 (정리 완료 기준)

```
project-root/  
  admin/  
    index.html  
    server_quali.py  
    assets/      # 정적
```

```
dist/          # 번들
auth_utils.py
db.py
__init__.py
tools/         # 스크립트/유틸
archive/, data/, logs/, scripts/, docs/
Dockerfile
.gcloudignore
.dockerignore
.env          # 로컬 전용 (루트에만 1개)
orchestrator.py
requirements.txt
config.json    # (있다면)
```

admin/.env 사용 금지. server_quali.py가 루트 .env만 읽음(LoadDotenv).
비밀키/토큰은 **Secret Manager** 또는 **환경변수**로 주입(리포 커밋 금지).

1.2 Cloud Run 서비스 확인(PS)

```
$SERVICE="quali-journal-admin"    # 실제 서비스명으로 교체 가능
$REGION = "asia-northeast3"        # 서울
$URL = (gcloud run services describe $SERVICE --region $REGION --format
"value(status.url)")
"$SERVICE / $REGION / $URL"
```

1.3 운영 헬스체크 & KPI

```
Invoke-WebRequest "$URL/health" -UseBasicParsing
$TOKEN = (gcloud secrets versions access latest --secret=ADMIN_TOKEN |
Out-String).Trim()
Invoke-WebRequest "$URL/api/status" -Headers @{ "X-Admin-Token" = $TOKEN }
-UseBasicParsing
```

2) 프론트엔드 패치 사항(최종본)

2.1 새로고침 버튼 먹등 바인딩 (중복 제거 버전)

index.html의 &/body& 바로 위에, 한 번만 존재해야 하는 블록:

```
&!-- === Bind: refresh buttons (idempotent) === --&
&script id="bind-refresh-buttons"&
(function(){
  function safe(fn){ return (typeof fn === 'function') ? fn : function(){}; }
```

```

function wire(id, handler){
  var el=document.getElementById(id);
  if(!el || el.dataset.wired==='1') return;
  el.addEventListener('click', function(ev){
    ev.preventDefault();
    try{
      if (typeof window.runWithUiFeedback==='function'){ runWithUiFeedback(el,
handler); }
      else { handler(); }
    }catch(e){ console.error('[bind-refresh]', e); }
  }, false);
  el.dataset.wired='1';
}
document.addEventListener('DOMContentLoaded', function(){
  wire('btnRefresh', safe(window.loadReady)); // Ready 새로고침
  wire('btnKpiRefresh',safe(window.refresh)); // KPI 새로고침
});
})();
</script>

```

중간에 있던 다른 DOMContentLoaded 바인딩은 초기 1회 로드만 남기고 삭제:

```

<script>
document.addEventListener('DOMContentLoaded', function(){
  try{ if(typeof refresh==='function'){ refresh().catch?(()=>=&{});} }catch(_){}
  try{ if(typeof loadReady==='function'){ loadReady().catch?(()=>=&{});} }catch(_){}
});
</script>

```

2.2 syncFallback(kind, body) if-else 사다리 교정

해당 함수의 const st = {...};다음 줄부터,

if (st.status === 'running') st.status = 'done';**직전까지** 전체를 아래로 교체:

```

    if (kind === 'merged') {
      let a = await fetch(API_BASE + '/api/flow/daily', { method:'POST',
headers: authHeaders() });
      a = await a.json().catch(()=>&({ok:false,steps:[]}));
      mergeSteps(st, a); if (!a.ok) st.status = 'error';
      renderSteps(st); drawTimeline(st);

      let b = await fetch(API_BASE + '/api/flow/community', { method:'POST',
headers: authHeaders() });

```

```

    b = await b.json().catch(()=>({ok:false,steps:[]}));
    mergeSteps(st, b); if (!b.ok) st.status = 'error';

    } else if (kind === 'daily' || kind === 'community') {
        let r = await fetch(API_BASE + '/api/flow/' + kind, { method:'POST', headers:
authHeaders() });
        r = await r.json().catch(()=>({ok:false,steps:[]}));
        mergeSteps(st, r); if (!r.ok) st.status = 'error';

    } else if (kind === 'keyword') {
        let r = await fetch(API_BASE + '/api/flow/keyword', {
            method:'POST',
            headers: Object.assign({'Content-Type':'application/json'}, authHeaders()),
            body: JSON.stringify(body || {})
        });
        r = await r.json().catch(()=>({ok:false,steps:[]}));
        mergeSteps(st, r); if (!r.ok) st.status = 'error';

    } else if (kind === 'keyword-collect') {
        let r = await fetch(API_BASE + '/api/flow/keyword/collect', {
            method:'POST',
            headers: Object.assign({'Content-Type':'application/json'}, authHeaders()),
            body: JSON.stringify(body || {})
        });
        r = await r.json().catch(()=>({ok:false,steps:[]}));
        mergeSteps(st, r); if (!r.ok) st.status = 'error';

    } else {
        el('log').textContent = '알 수 없는 작업: ' + kind;
        return;
    }
}

```

2.3 게이트 슬라이더 - 즉시 반영 + 저장 키 교정

즉시 반영(oninput): 숫자뿐 아니라 gatePass도 갱신

&input id="gateSlider" type="range" min="1" max="30"

oninput="

```

(function(el){
    var v=Number(el.value||0);
    var gv=document.getElementById('gateVal'); if(gv) gv.textContent=v;
    var apEl=document.getElementById('selApproved');
    var approved=Number((apEl&&apEl.textContent||'0').replace(/^[^0-9.-]/g,''))||0;

```

```

    var pass=document.getElementById('gatePass');
    if(pass) pass.textContent=(approved&=v?'통과':'미통과');
  })(this)"
  onchange="updateGate(this.value)"&

```

저장(PATCH) 바디: 함수 내부를 {"gate_required": n}로 수정

```

async function updateGate(v){
  const body = JSON.stringify({ gate_required: Number(v) });
  await fetch(API_BASE + '/api/config/gate_required', {
    method: 'PATCH',
    headers: Object.assign({'Content-Type':'application/json'}, authHeaders()),
    body
  });
  refresh(); // 저장 후 KPI 재조회
}

```

2.4 “수집만” 버튼 동작

프론트 요청: /api/flow/keyword + { collect_only:true }

```

async function runKeywordCollectOnly(keyword){
  const body = { keyword: keyword || getKeywordFromUI(), collect_only: true };
  return fetch(API_BASE + '/api/flow/keyword', {
    method:'POST',
    headers: Object.assign({'Content-Type':'application/json'}, authHeaders()),
    body: JSON.stringify(body)
  });
}

```

(선택) 서버 별칭 라우트 추가 원하면 /api/flow/keyword/collect → 위 요청으로 프록시.

3) 백엔드/배포

3.1 Dockerfile (요지: 오케스트레이터/정적 포함 보장)

```

FROM python:3.11-slim
ENV          PYTHONDONTWRITEBYTECODE=1          PYTHONUNBUFFERED=1
PIP_DISABLE_PIP_VERSION_CHECK=1
RUN apt-get update && apt-get install -y --no-install-recommends curl
ca-certificates && rm -rf /var/lib/apt/lists/*
WORKDIR /app

```

```
COPY requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r /app/requirements.txt || true

COPY orchestrator.py /app/orchestrator.py
COPY admin/ /app/admin/
COPY tools/ /app/tools/
COPY config.json /app/config.json
RUN mkdir -p /app/archive /app/logs
EXPOSE 8080
ENV PORT=8080
CMD ["python", "-u", "admin/server_quali.py"]
```

3.2 .gcloudignore / .dockerignore (핵심만)

.dockerignore

```
**/__pycache__/  
**/.pytest_cache/  
**/.git/  
**/.venv/  
*.pyc  
*.pyo  
.env  
*.log  
# (admin/, tools/, orchestrator.py 등 앱 소스는 절대 제외 금지)
```

.gcloudignore

```
**/.git/  
**/.venv/  
**/__pycache__/  
**/.pytest_cache/  
.env
```

```
!orchestrator.py  
!admin/**  
!tools/**  
!config.json  
!requirements.txt  
!Dockerfile  
!.github/**
```

3.3 배포/트래픽(PS)

```
gcloud run deploy $SERVICE --region $REGION --source . --allow-unauthenticated
--traffic latest=100
gcloud run services update-traffic $SERVICE --region $REGION --to-latest # 안전
망
```

4) 스모크 테스트(브라우저/PS 공용)

브라우저(DevTools Network 컨 상태)

KPI 새로고침 → GET /api/status 200

Ready 새로고침 → GET /api/items?state=ready 200

수집만 → POST /api/flow/keyword + {"collect_only":true} 200

키워드 특집(전체) → POST /api/flow/keyword 200

Merged → POST /api/flow/daily 이어 POST /api/flow/community 200

게이트 → 드래그 즉시 레이블 변경, **놓으면** PATCH /api/config/gate_required 200 → 자
동 KPI 재조회

PowerShell(요약)

```
Invoke-WebRequest "$URL/api/items?state=ready" -Headers @{ "X-Admin-Token" =
$TOKEN }
```

```
Invoke-WebRequest "$URL/api/flow/keyword" -Method POST -Headers @{
"X-Admin-Token" = $TOKEN } `
```

```
-ContentType "application/json" -Body '{"keyword":"IPC-A-610","collect_only":true}'
```

```
Invoke-WebRequest "$URL/api/flow/keyword" -Method POST -Headers @{
"X-Admin-Token" = $TOKEN } `
```

```
-ContentType "application/json" -Body '{"keyword":"IPC-A-610"}'
```

```
Invoke-WebRequest "$URL/api/flow/daily" -Method POST -Headers @{
"X-Admin-Token" = $TOKEN }
```

```
Invoke-WebRequest "$URL/api/flow/community" -Method POST -Headers @{
"X-Admin-Token" = $TOKEN }
```

```
Invoke-WebRequest "$URL/api/config/gate_required" -Method PATCH -Headers @{
"X-Admin-Token" = $TOKEN } `
```

```
-ContentType "application/json" -Body '{"gate_required":15}'
```

5) 트러블슈팅 체크리스트

증상
/api/status 401 / "invalid or missing
token"

빠른 원인 가설
헤더 누락

즉시 조치
헤더 X-Admin-Token: &값&
기)

/health 404	엔드포인트는 /health (토큰 불필요) 경로 확인, 서비스 URL 재확인
"orchestrator.py not found"	이미지에 파일 미포함 Dockerfile COPY orchestrator
새 UI가 안 보임	제 포함 Cloud Run 트래픽 최신 리비전
슬라이더 저장 실패	{"gate_required": n} 로 전송
"수집만" 404	잘못된 경로 /api/flow/keyword + {collect_
	/api/flow/keyword/collect

6) 다음 채팅을 위한 “시드 문장”

이 문장을 새 채팅 첫 메시지로 붙여주세요.

“퀄리저널 Admin 작업 연속성 확보를 위해 인수인계 보고서를 붙입니다. 현재 Cloud Run에 quali-journal-admin(asia-northeast3) 서비스가 배포되어 있고, /health 200, /api/status 200(X-Admin-Token) 확인했습니다. 프론트는 (1) ‘수집만’ 버튼을 /api/flow/keyword + {collect_only:true}로 수정, (2) updateGate() 바디를 {"gate_required":n}로 보내도록 정리해야 합니다. 버튼 먹등 바인딩/syncFallback 사다리/게이트 oninput 즉시 반영은 적용되어 있습니다. Dockerfile은 orchestrator/ admin/ tools/ 포함 COPY, .gcloudignore/.dockerignore는 필수 소스 포함 규칙으로 구성되어 있습니다. 이 상태에서 남은 프론트 2건 마무리 및 스모크(6종) 점검을 이어가 주세요.”

필요하면, 위 코드 블록을 기준으로 우리가 바꾼 정확 위치를 다시 지정해 드리거나, 새 배포가 반영되지 않을 때 로그/권한 탭에서 무엇을 봐야 하는지 바로 이어서 안내해 드리겠습니다.